Darpan Patel

SHRINAV WORKSHOP

Practice Assignment

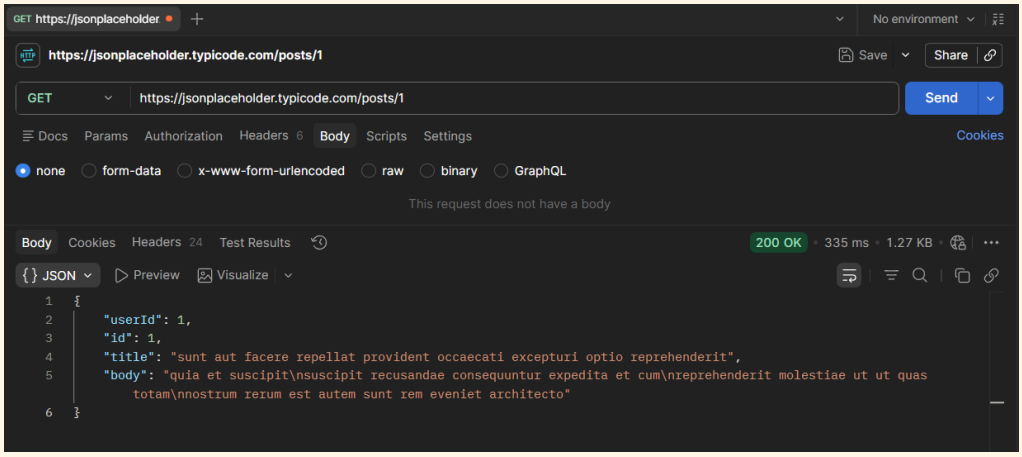
API Fundamentals & Postman Mastery

Api for every task: https://jsonplaceholder.typicode.com

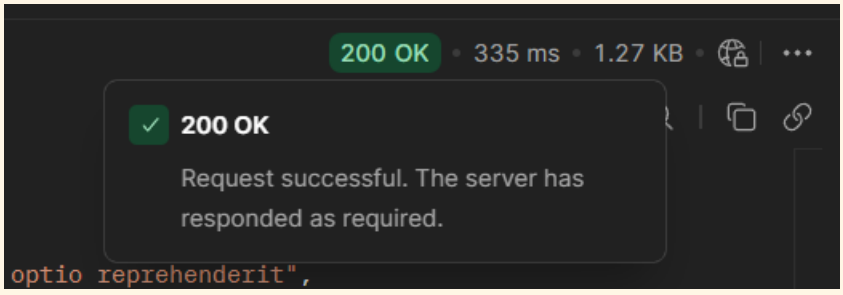
Tasks

TASK 1 Reading Data Without Headers

a) Send a GET request to /posts/1 — leave the Headers and Body tabs completely empty.



b) Record the status code you get back.

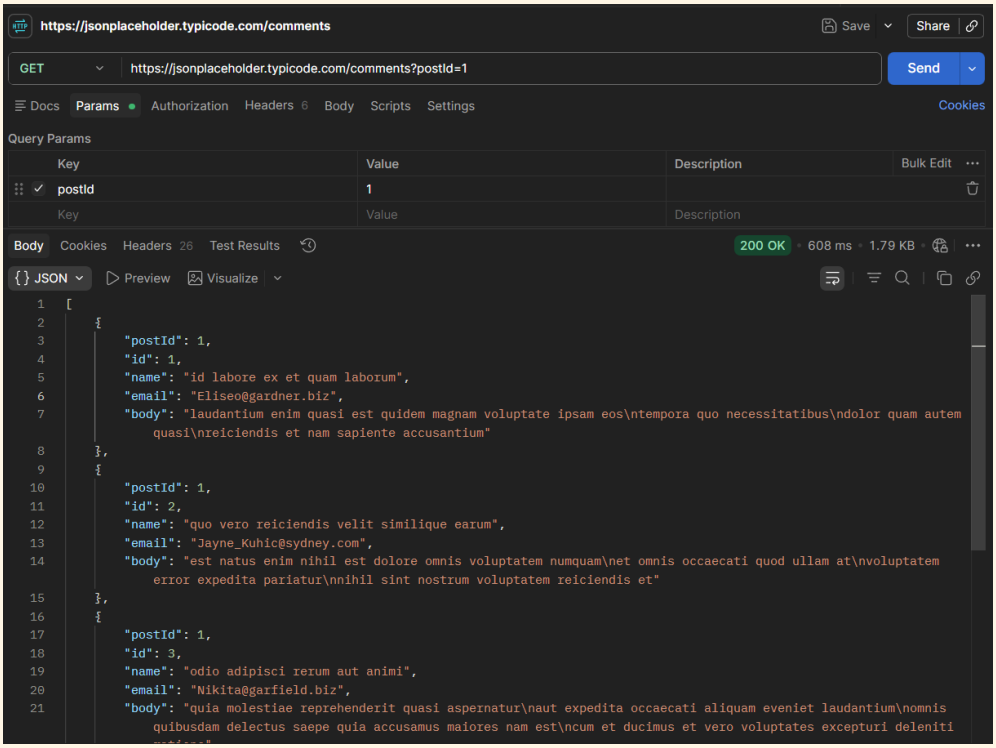


c) In one sentence, explain why this request works even with nothing in the Headers tab.

⇨ This request works without headers because the JSONPlaceholder API allows public GET requests. It returns the requested data without requiring authentication or additional header information.

TASK 2 Filtering With Query Parameters

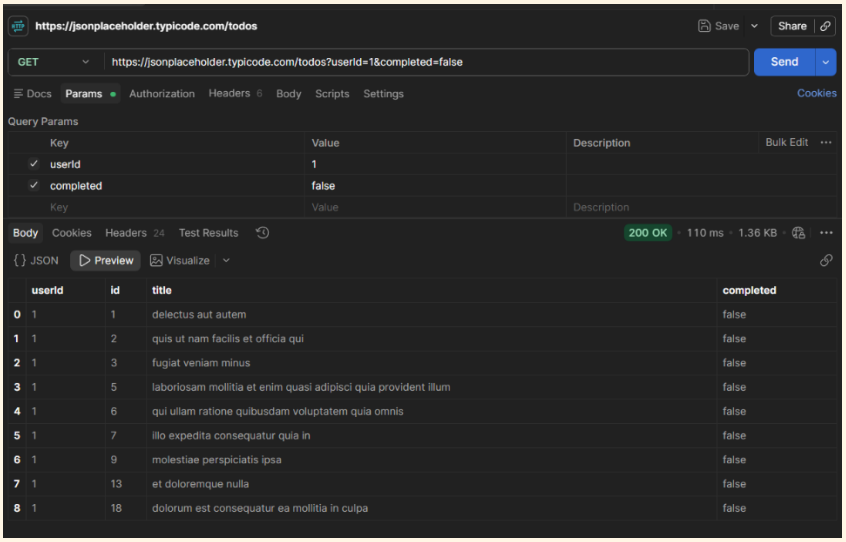
a) Send a GET request to /comments, and add a query parameter in the Params tab: postId = 1.



b) Confirm the URL automatically updates to include ?postId=1.

⇨ Yes, Postman automatically updated the URL to include ?postId=1 after adding the query parameter in the Params tab.

c) Now combine two filters at once on /todos: userId = 1 and completed = false.

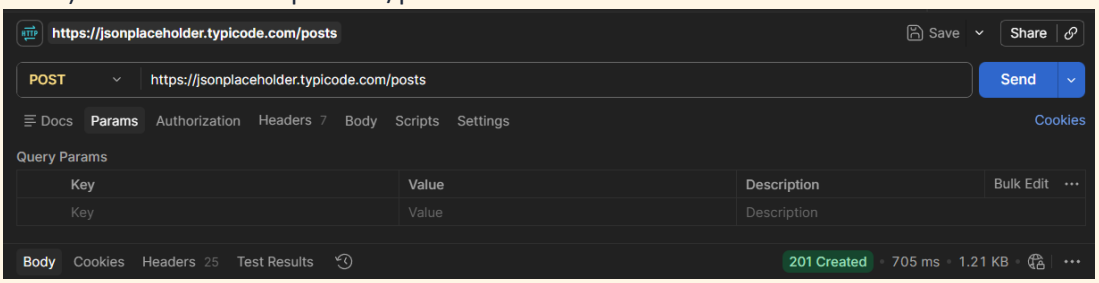


d) In one sentence, explain what changed in the results compared to not using any params.

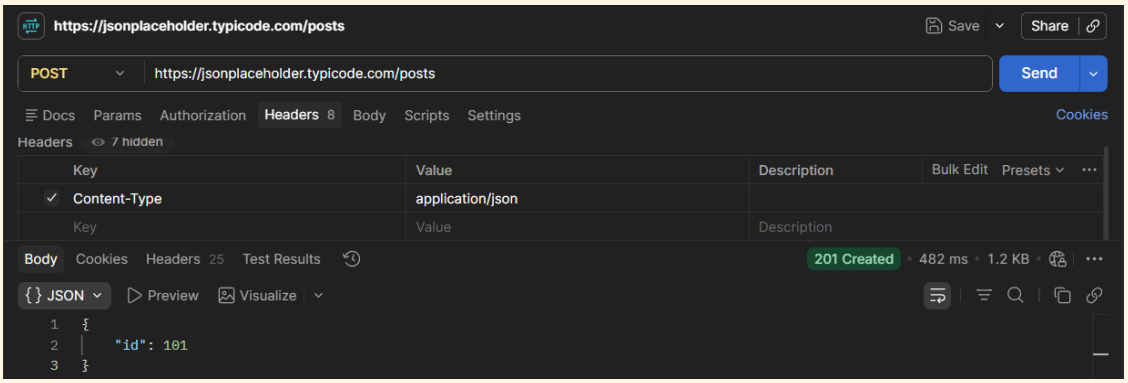
⇨ Using query parameters limits the response to only the matching records. Without query parameters, the API returns all available data, but with filters, it returns only the relevant results.

TASK 3 Sending Data With Headers & a Body

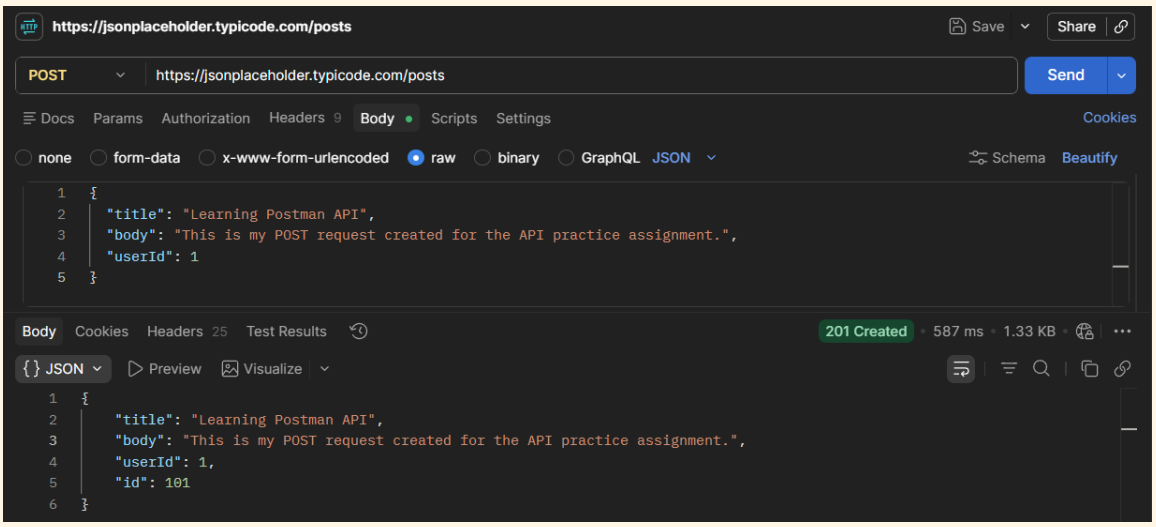
a) Send a POST request to /posts.



b) In the Headers tab, add: Content-Type: application/json



c) In the Body tab (raw, JSON), send a new post with a title, body, and userId of your choice.



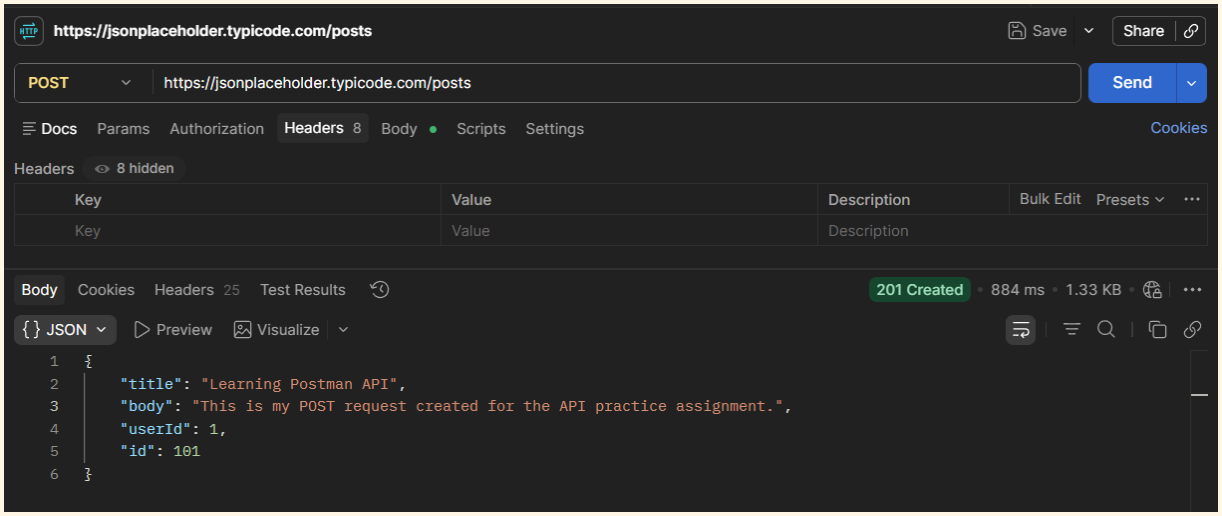
d) Record the status code and the new id the server assigned.

⇨ Status Code: 201 Created

⇨ New ID Assigned: 101

TASK 4 Headers vs. No Headers

a) Repeat Task 3's POST request, but this time remove the Content-Type header before sending.



b) Compare the response to Task 3 — is it different? Record what you observe.

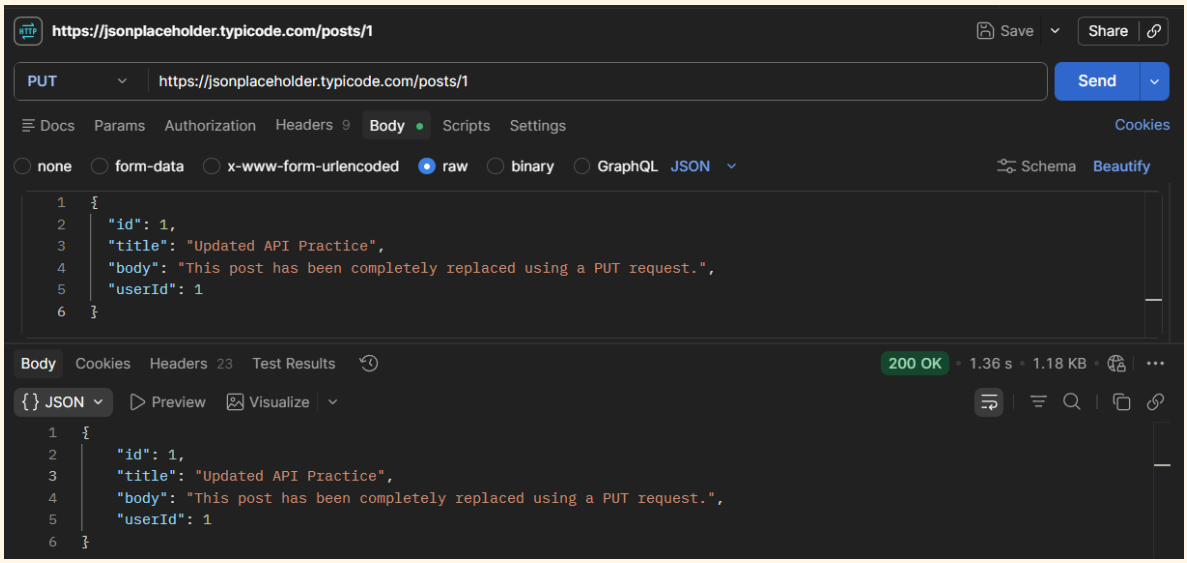
With Content-Type	Without Content-Type
Status: 201 Created	Status: 201 Created
Request processed	Request still processed

c) In 2–3 sentences, explain why headers matter for a request like this, even though this practice API is forgiving about it.

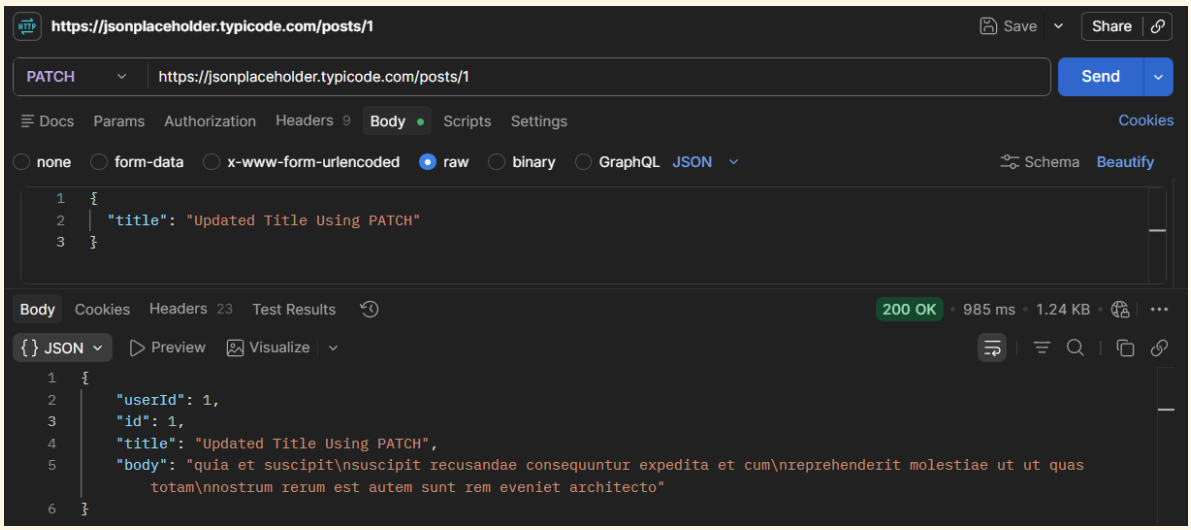
⇨ Headers help the server understand how the request data should be processed. The Content-Type: application/json header tells the server that the request body is in JSON format. Although JSONPlaceholder accepts the request without this header, many real-world APIs may return an error if the correct header is missing.

TASK 5 PUT vs. PATCH

a) Send a PUT request to /posts/1 that replaces the entire post (include id, title, body, and userId in the body).



b) Send a PATCH request to the same /posts/1 that changes only the title.



c) In one sentence, explain the difference between what PUT sent and what PATCH sent.

⇨ A PUT request replaces the entire resource, so all fields must be included in the request body. A PATCH request updates only the specified field(s), making it suitable for partial changes.

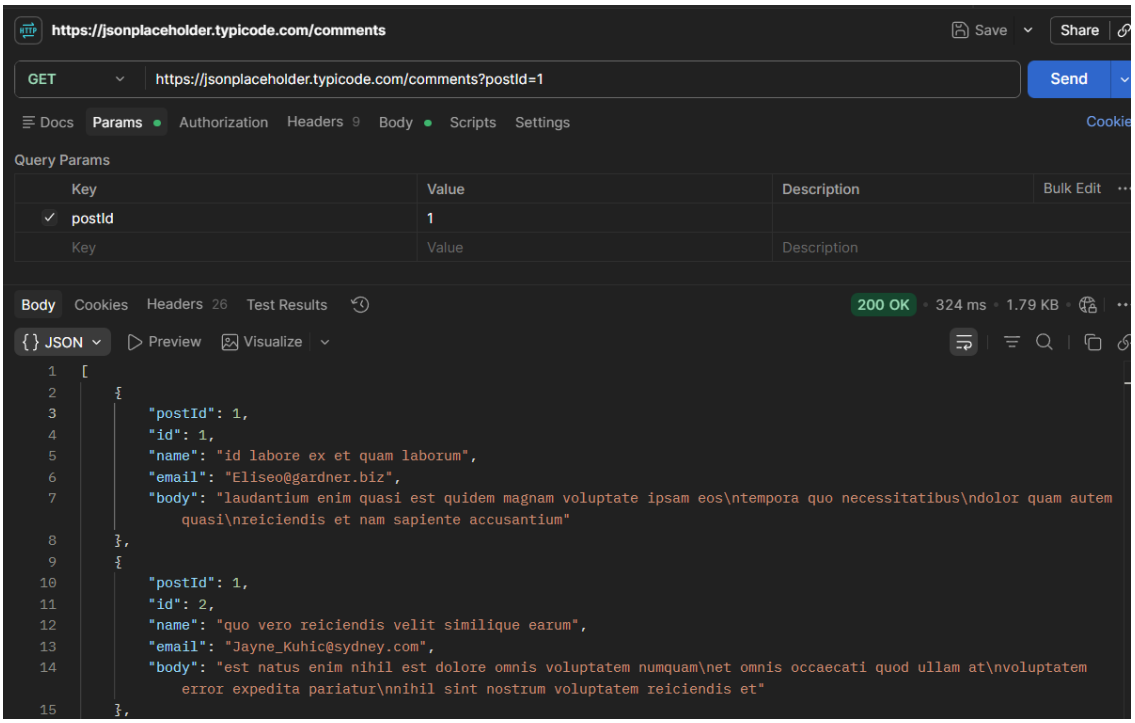
Final Challenge — A Realistic Request Flow

TASK 6 **The Mini Feedback System**

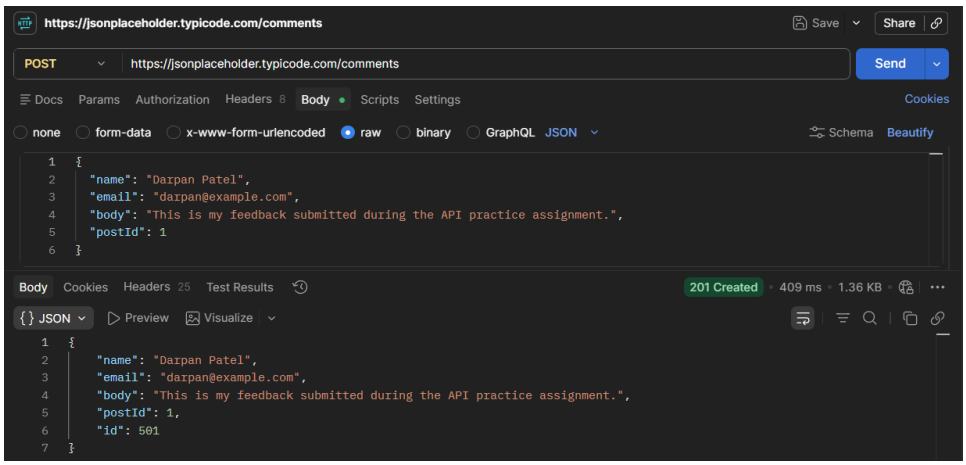
Scenario: you're testing the backend for a simple feedback form, where users read existing comments on a post and can leave their own.

Complete the following steps, in order:

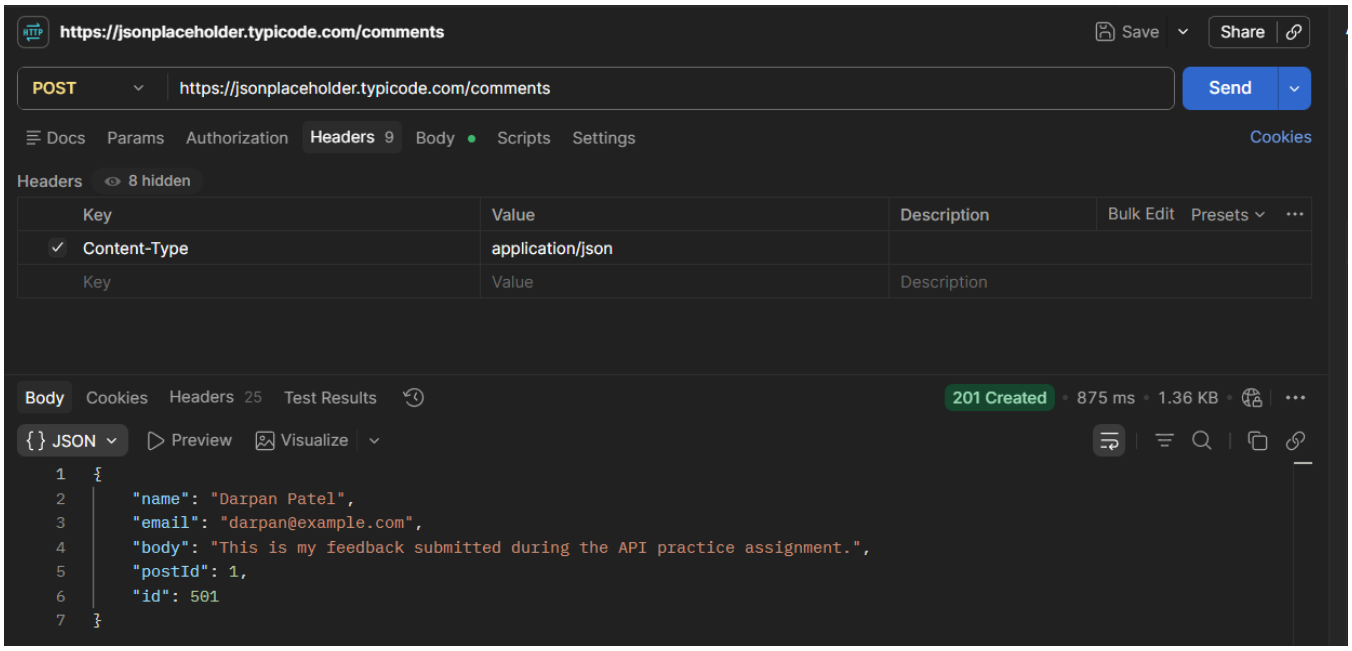
- a) Send a GET request to /comments, using a query parameter to fetch only the comments for postId = 1.



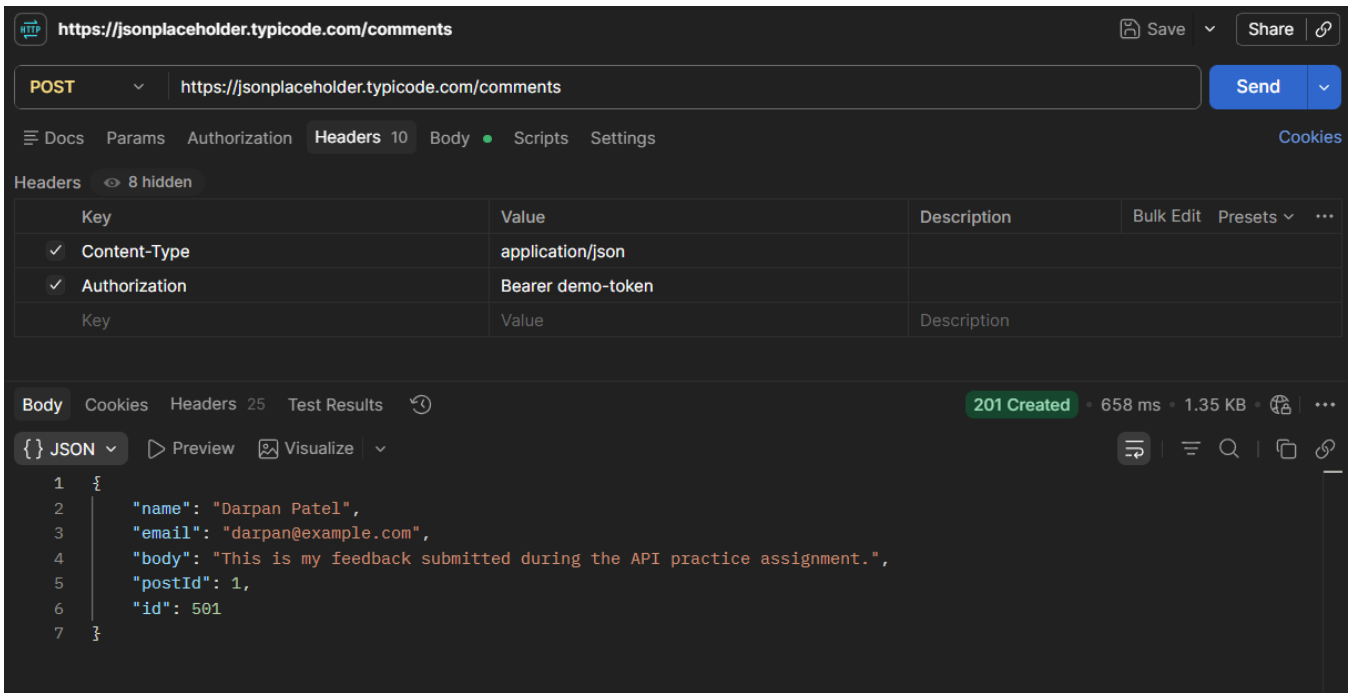
- b) Send a POST request to /comments to add a new comment — with no headers at all. Record the status code.



- c) Resend the same POST from step (b), this time with the Content-Type header included. Compare the two responses.



- d) Add one more header to this request: Authorization: Bearer demo-token — to simulate a login-protected submission, even though this practice API won't actually check it.



- e) Record the status code for every step above (a–d) in a short table or list.

Step Request	Status Code
(a) GET /comments?postId=1	200 OK
(b) POST /comments (No Headers)	201 Created
(c) POST /comments (Content-Type)	201 Created
(d) POST /comments (Content-Type + Authorization)	201 Created

- f) In 2–3 sentences, explain what would happen differently at step (d) if this were a real API that actually required authentication.

⇨ If this were a real API, the server would check the Authorization token before accepting the request. A valid token would allow the request to be processed, while an invalid or missing token could result in errors such as 401 Unauthorized or 403 Forbidden.